

# CrewAI Multi-Agent Collaboration — Roles & Task Delegation

M. Qasim | Week 2, Day 4 Assignment | Due: 10 Sept 2026

## 1. Multi-Agent Design

**Chosen task:** research a competitor, summarize findings, and draft a marketing angle — specifically, research three CRM competitors (HubSpot, Salesforce, Zoho) and produce a fact-checked marketing blog post comparing our product against them.

| Agent | Role                     | Goal                                                                                              | Backstory                                                                                                                       |
|-------|--------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| A     | Senior Market Researcher | Find accurate, up-to-date pricing, features, and positioning for leading CRM competitors          | 10 years in competitive intelligence for B2B SaaS; distrusts marketing copy and checks primary sources before repeating a claim |
| B     | Content Strategist       | Turn validated research into a compelling, on-brand marketing blog post                           | Former journalist turned SaaS content strategist; turns dry facts into a story without losing accuracy                          |
| C     | Fact-Checker & Editor    | Verify every claim in the draft against the source research and polish it for a business audience | Ex-newspaper copy editor with zero tolerance for unverified claims or clunky prose                                              |

No two agents own the same responsibility: the Researcher is the only one that touches raw competitor data, the Writer is the only one that produces prose, and the Editor is the only one that verifies and polishes. Each hands a work product to the next rather than redoing the previous step.

**Why specialization helps here.** A generalist agent doing research, writing, and editing in one pass tends to skip the adversarial step — a model that just wrote a claim has little incentive to doubt it, so unverified claims slip through more easily than when a separate agent's entire job is to distrust the draft. Splitting the roles also keeps each agent's prompt narrow and on-task instead of balancing "research carefully" against "write persuasively" against "police your own claims" at once. **Where it breaks down:** for a genuinely small, single-skill task, three agents add coordination overhead — extra calls, latency, cost — for no quality gain over one well-prompted agent. Multi-agent crews pay off when the task has multiple distinct skills and is valuable enough to justify an internal checks-and-balances process.

## 2. Tool Assignment

| Agent                 | Tool(s)                                                 | Why                                                                                                                                                                 |
|-----------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Researcher            | Competitor Intel Lookup (custom)                        | The only agent that needs raw competitor data; a local dataset stands in for a live web-search tool (no SerperDevTool / network access in this offline environment) |
| Content Strategist    | Read a file's content (FileReadTool, real crewai_tools) | Reads brand_voice_guidelines.md for tone rules — not a duplicate of the researcher's job, since research already arrives via context=[research_task]                |
| Fact-Checker & Editor | Readability & Claim Check (custom)                      | A light verification tool: flags word count and unverifiable superlatives ("best", "guaranteed", "#1") rather than redoing the researcher's job                     |

**Offline-tooling note:** the assignment's example code uses SerperDevTool, which needs a SERPER\_API\_KEY and live internet search. This sandbox has neither a GEMINI\_API\_KEY nor outbound access to Google/Serper, so the Researcher's tool is a small local competitor dataset instead — the same "plain Python tool reading a small local data source" pattern used for knowledge\_base.py in the Day 3 assignment. Swapping in SerperDevTool() (with a SERPER\_API\_KEY set) is a one-line change; nothing else in the crew needs to change.

### 3. Sequential vs. Hierarchical

Two versions of the research task's *expected\_output* were tested under `Process.sequential`: a loose v1 ("summarize what you found") and a tight v2 (fixed labeled sections: Competitor Name / Pricing / Key Features / Market Position). The loose version let the Researcher return one unstructured paragraph mixing all three competitors, which gave the Writer nothing to reliably index into — its blog post came out generic, with no real pricing figures quoted. Tightening the *expected\_output* fixed this: the resulting post cited concrete numbers ( \$20/seat/month, \$14–\$52/user/month, the Zia AI assistant) grounded directly in the labeled research fields. The same tight tasks were then re-run under `Process.hierarchical` with an auto-created Crew Manager delegating each task to the correct coworker via CrewAI's builtin Delegate work to coworker tool.

| Aspect        | Sequential                                                     | Hierarchical                                                                                                                                                                               |
|---------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quality       | Same final post, produced directly by the 3 specialist agents  | Identical final post — the manager only relays each coworker's completed output                                                                                                            |
| Latency       | Lower — 3 agent turns, no delegation overhead                  | Higher — every task adds a manager delegate-thenrelay round trip on top of the same 3 agent turns                                                                                          |
| Cost (tokens) | Lower — ~21.5K total tokens for the tight run (see Section 4)  | Higher — ~124,560 total tokens: the manager's system prompt (tool schemas for every coworker, plus delegate/ask-question tools) is added on top of the 3 agents' own prompts               |
| Reliability   | High — fixed, predictable agent-to-agent handoff order         | High here too, but only because the manager's one job per task is "delegate to the pre-assigned agent"; with more agents or ambiguous ownership, a real manager LLM can misroute or dither |
| When to use   | Task ownership per step is already known and fixed (this task) | Task ownership varies at runtime, needs re-planning, or a task should bounce between coworkers mid-run                                                                                     |

For this task — a fixed 3-step pipeline where each step's owner is known in advance — hierarchical adds cost and latency without adding anything sequential doesn't already provide. It would earn its keep on a task where the right next agent isn't decidable ahead of time.

### 4. Cost & Evaluation

**Live API run caveat:** This run used the **live Gemini API** (model: gemini-3.5-flash-lite) because a `GEMINI_API_KEY` was available in the environment. Token counts are therefore from the actual provider tokenizer, and latency includes real network and model generation time. The figures below are accurate for estimating real-world cost and performance, but note that the free tier's rate limits (5 requests per minute) caused `max_rpm=5` throttling and a 60-second sleep between runs. The sequential-vs-hierarchical gap is substantial: hierarchical uses ~4.6x more total tokens and 3x more requests than sequential, and is ~27% slower in wall-clock time. These numbers can be used to price against Gemini's actual per-token rate (input: \$0.10 per 1M tokens, output: \$0.40 per 1M tokens for Flash Lite)..

| Run                   | Prompt tokens | Completion tokens | Total tokens | Requests | Latency |
|-----------------------|---------------|-------------------|--------------|----------|---------|
| Sequential v1 (loose) | 25,428        | 8,094             | 33,522       | 24       | 78.74 s |
| Sequential v2 (tight) | 20,433        | 6,864             | 27,297       | 24       | 75.63 s |
| Hierarchical (tight)  | 98,248        | 26,312            | 124,560      | 72       | 96.47 s |

Compared with Day 3's single-agent LangGraph solution (one LLM per node, ~5–6 calls per full run including self-correction), this crew's **sequential** run uses roughly **4x** as many LLM calls (24 requests vs ~6), and **hierarchical** roughly **12x** as many (72 requests vs ~6) — more specialized agents means more total LLM round trips for the same task, the direct cost of the reliability benefit multi-agent designs are meant to buy. The token gap is even wider: sequential v2 used **27,297** total tokens, hierarchical used **124,560** total tokens (a **4.6x** increase), while a single-agent LangGraph run would typically consume only ~8–10K tokens for comparable work. In short: sequential is already ~3x more expensive than a single agent in tokens, and hierarchical multiplies that again by ~4.6x, making the manager overhead the single biggest cost driver in this comparison.

**Success criteria (manually scored, 1–5)**

| Criterion                                                                             | Sequential v1 (loose)                                            | Sequential v2 (tight)                                       | Hierarchical (tight)         |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------|------------------------------|
| Factual grounding — every price/feature claim traceable to the research tool's output | 3 — claims true but too vague to verify against specific numbers | 5 — every number matches a labeled research field           | 5 — same grounded post as v2 |
| Completeness — headline, intro, percompetitor paragraph, CTA, fact-check notes        | 4 — all sections present but competitor paragraphs are thin      | 5 — all sections present and substantive                    | 5 — same output as v 2       |
| Tone — appropriate for a business buyer, no unverified superlatives                   | 4 — fine, if generic                                             | 5 — confident, numbersfirst, matches brand voice guidelines | 5 — same output as v 2       |

Sequential v1 and v2 use the same 3 scripted agents on the same underlying facts, so the score gap above is entirely attributable to the expected\_output fix, not to any difference in the agents or tools. Hierarchical scores match v2 because the manager only relays the same coworkers' work.

### Was the crew worth it?

Yes, in the sequential configuration: the built-in "someone else checks the last person's work" structure that comes free with 3 agents caught a real, concrete win (the expected\_output fix in Section 3) that a single generalist prompt likely would have needed a manual review pass to catch. Hierarchical was **not** worth its added cost and latency here, since task ownership was fixed and known ahead of time — every extra token it spent went into the manager relaying, not improving, the same work sequential already produced.

Specialization was worth it; paying for a manager's added coordination was not, for a pipeline that doesn't need runtime re-planning.

---

Full implementation, execution logs, and token-usage output: Day4\_CrewAI\_MultiAgent.ipynb